

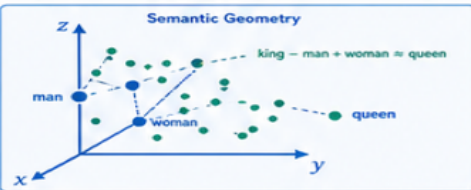
EMBEDDINGS

1 WHY

Turn messy data into meaningful geometry.

- Capture SEMANTICS:** similar meaning $\rightarrow$ nearby vectors
- Enable SCALE:** retrieve, cluster, classify at billion-item scale
- Make it USEFUL:** RAG, search, personalization, recommendations, dedupe

Goal: place related things close in vector space.



4 COMPLEXITY

Costs at query time (N = # vectors, d = dims)

Similarity Compute (per query)

Dot / Cosine	$O(N \cdot d)$	$\sim 2 \cdot N \cdot d$ FLOPs
L2 Distance	$O(N \cdot d)$	$\sim 2 \cdot N \cdot d$ FLOPs

Storage

N vectors	$N \times d \times B$ bytes	(B = bytes/float)
float32 (4B)	$4 \cdot N \cdot d$	
float16 / bfloat16 (2B)	$2 \cdot N \cdot d$	
int8 quantized (1B)	$1 \cdot N \cdot d$	

Example: $N = 100M$

d	128	384	768
float32	51 GB	154 GB	308 GB
int8	12.8 GB	38.4 GB	76.8 GB

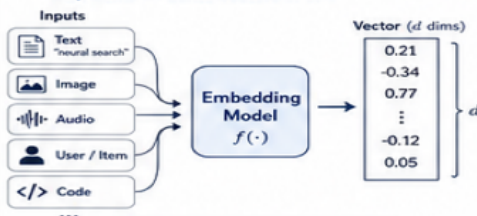
ANN Index Build (rough)

HNSW	$O(N \log N)$	Memory: $\sim M \cdot N$
IVF	$O(N)$	+ training: $O(N \cdot d)$
PQ	Adds codebook: $O(K \cdot d)$	

Cost drivers: N , d , QPS, index type, freshness.

2 CORE IDEA

Map items $\rightarrow$ dense vectors in $\mathbb{R}^d$.

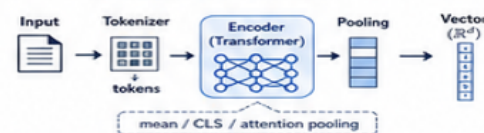


- Properties we want
- ✓ Semantically similar $\rightarrow$ close
 - ✓ Robust to wording / noise
 - ✓ Smooth neighborhoods
 - ✓ Useful across tasks

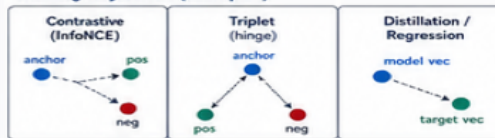
Key levers: model, dimension (d), objective, data.

3 INTERNALS

How embeddings are produced.



Training Objectives (examples)



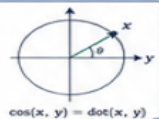
Typical d : 128–1536 (depends on model & use case)

5 TRADE-OFFS

Trade-Off	Option A	vs	Option B	Impact
Accuracy	Higher d		Lower d	$\uparrow$ recall $\uparrow$ cost
Latency	Exact search $\uparrow$ accuracy		ANN search $\downarrow$ latency	$\uparrow$ accuracy $\downarrow$ accuracy
Freshness	Real-time updates		Batch updates	$\uparrow$ cost/infra $\downarrow$ cost
Memory	float32 $\uparrow$ memory		Quantized (int8 / PQ)	$\uparrow$ memory $\downarrow$ accuracy
Simplicity	Brute force		Complex index	harder ops better scale

Normalization

- Necessary for COSINE similarity
- L2-normalize vectors: $v' = \frac{v}{\|v\|_2}$
- After normalization: $\text{dot}(x, y) = \cos(x, y)$



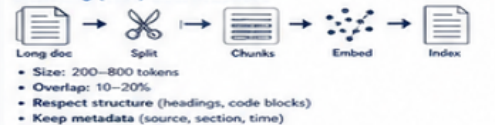
Rule of thumb: Start simple $\rightarrow$ measure $\rightarrow$ optimize the bottleneck.

6 DISCUSSION

Similarity Metrics

Cosine Similarity	$\text{sim} = \frac{x \cdot y}{\ x\ \ y\ }$	Best for direction (when normalized)
Dot Product	$\text{sim} = x \cdot y$	Works well when magnitudes matter
L2 Distance	$d = \ x - y\ _2$	Metric space, intuitive geometry

Chunking (Text) Best Practices



Retrieval Flow (RAG)



7 RELATED

Alternatives / Complementary

- BM25 / Lexical Search**: Exact term matching, transparent
- Hybrid Search**: BM25 + Embeddings (best of both)
- Cross-Encoders**: Rerank with deep pairwise models
- Sparse Embeddings**: SPLADE, BGE-M3 (sparse + dense)
- Graph / Knowledge Graphs**: Better for strict relations
- Traditional ML Features**: Tabular / signals for ranking

When to use what?

Keyword exact match	BM25
Semantic understanding	Embeddings
High precision ranking	Cross-Encoder rerank
Complex relations	Graphs
Mixed workloads	Hybrid

8 PRODUCTION

What can go wrong

- Embedding Drift**: Model/data changes shift vectors
- Poor Recall**: ANN params too aggressive
- Garbage In**: Noisy / low-quality input
- Domain Mismatch**: Out-of-distribution queries
- Cold Start**: New items with no embeddings
- Skewed Data**: Popularity bias in results

Key Practices

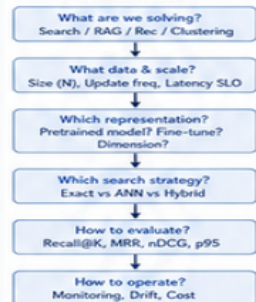
- Monitor Drift**: Track centroid shift, nearest-neighbor consistency
- Evaluate Recall@K**: Holdout Q&A set, track over time
- Quality Pipeline**: Clean, dedupe, normalize inputs
- Right Index & Params**: Tune HNSW (M, efSearch) or IVF (nlist, nprobe)
- Version Everything**: Model, index, data, pipelines
- Refresh Strategy**: Batch or streaming re-embed

Evaluate with:



9 INTERVIEW THINKING

Decision Flow



Ask clarifying questions

- What's the success?
- What's the data like?
- What's the scale (N , QPS, freshness)?
- Latency & cost constraints?
- Any filters / access controls?
- Offline eval data?
- Failure tolerance?

Communicate

- Define trade-offs
- Propose options
- Back-of-envelope math
- Iterate & de-risk

Pick a path, justify it, measure, iterate.

10 SENIOR NOTES

- Model Choice**: Start with strong foundation models
- Domain Adaptation**: Contrastive fine-tuning on in-domain pairs
- Dimension as a Knob**: Increase d before adding complex infra
- Quantization**: Start with int8 / PQ; measure quality drop
- Layer / Pooling Matters**: Try CLS vs mean vs last token
- System Design**: Index sharding, replication, backfills, blue/green
- Cost Discipline**: Track $\$/1K$ queries and storage growth

Senior heuristics

- 80% of quality comes from data & model.
- Don't over-index early.
- Measure business impact, not just offline metrics.
- Prefer simple, observable, recoverable systems.
- Automate evals & drift alerts.
- Keep a rollback plan.



11 MENTAL HOOK

Embeddings are a map.



Build a good map. Keep it fresh. Use the right compass.

Better map $\rightarrow$ better search $\rightarrow$ better outcomes.

Quick Reference

Normalize for cosine	$v' = \frac{v}{\ v\ _2}$
Cosine = dot (normalized)	$\cos(x, y) = x' \cdot y'$
Memory (floats)	$N \times d \times B$ bytes
FLOPs (dot)	$\sim 2 \times N \times d$
Chunk size	200–800 tokens
Overlap	10–20%
Good d range	128–1536

Clarity first. Define the problem.

Measure everything. Trust data.

Optimize the right thing.

Iterate, learn, improve.

VECTOR DATABASES

1 WHY

Rationale

- Store high-dimensional vectors + metadata
- Fast similarity search at scale
- Power semantic search, RAG, recs, dedup, anomaly detection, more

When to use

- Meaning is in proximity, not keywords
- Tolerate approximate results for big latency wins
- Need metadata filtering with vector search

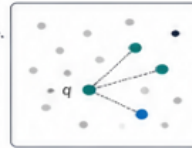
Key ingredients



2 CORE IDEA

Mental model

Map items to points in high-D space. Similar items are close. Return the nearest neighbors.



Distance / similarity

Lower distance = more similar

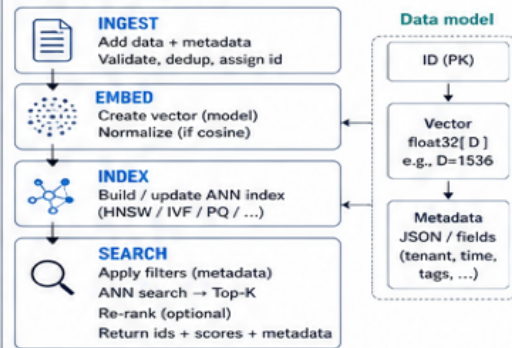
- Cosine similarity: $1 - \cos(u, v)$ (preferred)
- L2 (Euclidean): $\|u - v\|_2$
- Dot product: $-u \cdot v$ (or maximize $u \cdot v$)

Typical pipeline



3 INTERNALS

Lifecycle / query workflow



4 COMPLEXITY

Time / space (rules of thumb)

Index / Method	Build Time	Query Time (per query)	Memory (approx)
Flat (exhaustive)	$O(N \cdot D)$	$O(N \cdot D)$	N · D · 4B
HNSW	$O(N \cdot M \cdot \log N)$	$O(M \cdot ef)$	N · M · 4B
IVF (nlist)	$O(N \cdot D)$	$O(nprobe \cdot D)$	N · D · 4B
IVF + PQ (m bits)	$O(N \cdot D)$	$O(nprobe \cdot m)$	N · m · bits/8
DiskANN	$O(N \cdot \log N)$	$-O(\log N)$	Near disk

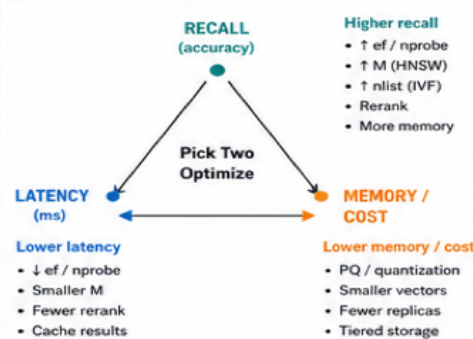
Legend

N = number of vectors, D = dimension, M = HNSW max links, ef = search breadth, nlist = #lists (IVF), nprobe = #lists probed, m = #subvectors, bits = PQ bits per subvector

Cost levers

↓ D (PCA) | ↓ precision (fp16 / int8) | ↓ M / ef / nprobe | Shard data

5 TRADE-OFFS



6 DISCUSSION

Choosing indexes

Index	Best for	Pros	Cons
HNSW	High recall, low latency	Excellent recall, fast queries	Higher memory, slower builds
IVF (Flat)	Large scale, balanced	Lower memory, tunable	Lower recall if nprobe small
IVF + PQ	Massive scale, memory bound	Big memory savings	Lower recall, needs tuning
DiskANN	Billion+, memory tight	Scales on disk, low RAM	Complex ops, build time

Alternatives / complements

- Keyword / BM25 search
 - Filters + SQL databases
 - Graph databases
 - Cache (Redis), Object storage for payloads
- Use alone or combine (hybrid search)

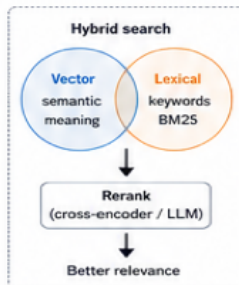
7 RELATED

Capabilities

- ✓ Similarity search (Top-K)
- ✓ Metadata filtering
- ✓ Range search (radius)
- ✓ Hybrid search (vector + lex)
- ✓ Aggregations / group by
- ✓ De-dup / clustering
- ✓ Multi-tenancy

Common indexes

- HNSW (graph-based)
- IVF (inverted file)
- PQ / OPQ (quantization)
- ScaNN, DiskANN, SPANN
- Brute force (Flat)



8 PRODUCTION

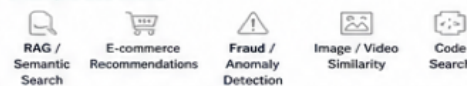
Operational checklist

- 📁 Ingestion pipeline (batch / streaming)
- 🔄 Idempotency + dedup
- 🔄 Backfills & re-indexing
- 📊 Monitoring (QPS, p95, recall, errors)
- 💾 Capacity: RAM / CPU / Disk
- 🔄 Backups & DR
- 🔒 Multi-tenancy isolation
- 🔒 Security: auth, encryption, row-level filters

Failure modes

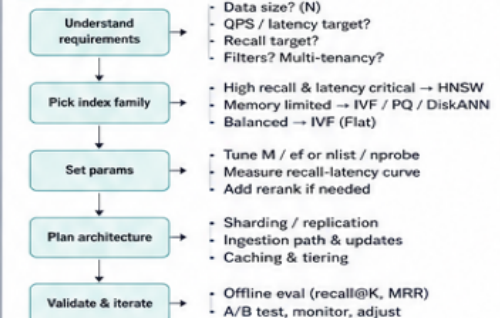
- ⚠️ Low recall -> missed results (bad params / index)
- ⚠️ High latency -> timeouts (hot partitions)
- ⚠️ Stale results -> old index not refreshed
- ⚠️ Filter mismatch -> empty results
- ⚠️ Memory pressure -> OOM, evictions
- ⚠️ Skewed tenants -> noisy neighbors

Example use cases



9 INTERVIEW THINKING

Decision flow



10 SENIOR NOTES

Model matters

Embedding model & dimension drive quality & cost. Re-embed carefully.

Quantify everything

Always plot recall vs latency vs memory. No "best" setting.

Updates are expensive

Deletes & updates cost; batch and rebuild when possible.

Think systems

Sharding, isolation, backpressure, caching, and observability beat clever algorithms.

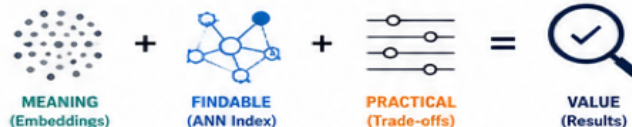
Correctness > cleverness

Filters first, then ANN, then rerank. Explain trade-offs.



11 MENTAL HOOK

"Vectors capture **MEANING**, indexes make it **FINDABLE**, trade-offs make it **PRACTICAL**."



Remember: Proximity ≠ Relevance. Use filters + reranking to close the gap.

Two questions to ask

- 1 What is the smallest change that improves recall the most?
- 2 What is the cheapest change that meets the SLA?

RAG

1 WHY



Ground LLMs in your data
Reduce hallucinations, increase trust



Keep answers up to date
Connect to sources that change



Improve relevance
Leverage rich, domain knowledge

RAG RATIONALE



MENTAL MODEL

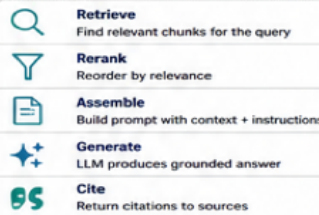
RAG = Memory for LLMs.
Find the right facts → show them to the model → get an answer with sources.

2 CORE IDEA

INGESTION (Offline)

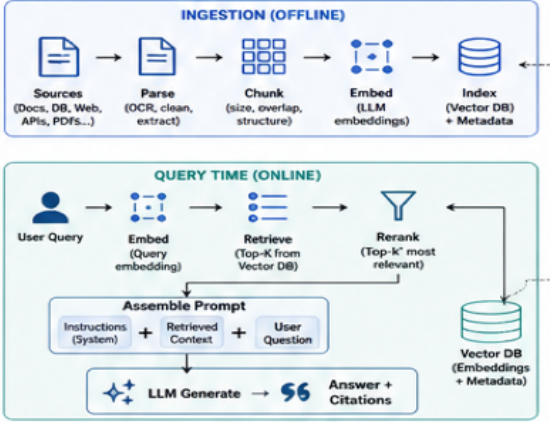


QUERY TIME (Online)



3 INTERNALS

INTERNAL WORKFLOW



TYPICAL SETTINGS

Chunk: 300-800 tokens | Overlap: 10-20% | Top-K: 5-20 | Rerank K*: 3-10

4 COMPLEXITY

TIME COMPLEXITY (per query)

Embedding (query): $O(d)$
Vector search (ANN): $\sim O(\log N)$
Rerank (cross-encoder): $O(K \cdot L)$
LLM generation: $O(T)$
 N = #chunks in index, d = embed dim, K = top-K, L = tokens per chunk, T = generated tokens

SPACE COMPLEXITY

$O(N \cdot d)$ for embeddings + metadata

COST DRIVERS (per 1K queries)

Embedding (query) \$ \$
Vector search \$
Rerank \$ \$ \$ \$
LLM generation \$ \$ \$ \$ \$

Optimize chunk size, K, and rerank usage to control cost.

5 TRADE-OFFS

DIMENSION	MORE OF THIS	LESS OF THIS
Chunk size	Bigger = more context less misses	Smaller = precise more overhead
Top-K retrieved	Higher recall	Higher cost, more noise
Reranking	Better relevance	Higher latency, cost
Freshness	More frequent updates	Staleness, more hallucinations
Metadata filters	Precision, safety	May miss relevant context
Prompt length	More context	Truncation, higher cost
Determinism	Constrained decoding, temperature ↓	More creative, less stable

Balance recall vs precision vs cost vs latency. There is no free lunch.

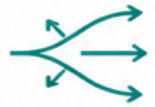
6 DISCUSSION

RAG vs LLM FINE-TUNING

Aspect	RAG	Fine-Tuning
Data freshness	High (update index)	Low (retrain)
Personalization	High (filters, metadata)	Medium
Hallucination reduction	High (with good context)	Medium
Cost to update	Low	High
Compute	Low-Med	High
Best for	Dynamic knowledge, documents, FAQs	Style, behavior, reasoning patterns

RELATED / ALTERNATIVES

- Fine-Tuning / Continued Pretraining
- Tool Use / Function Calling
- Knowledge Graphs
- Hybrid Search (BM25 + Vector)
- Long-Context LLMs



7 RELATED



Vector Databases: FAISS, Pinecone, Weaviate, Milvus, Qdrant



Embedding Models: OpenAI text-embedding-3, ES, BGE, Instructor



Rerankers: Cohere Rerank, BGE-reranker, Cross-Encoders



Chunkers: Recursive, Semantic, Markdown/AST



Eval Tools: RAGAS, Trulens, DeepEval, Helicone, LangSmith



Frameworks: LangChain, LlamaIndex, Haystack



Key Idea: Retrieval gives the model facts; RAG turns facts into grounded answers.

8 PRODUCTION



- ✓ **Monitoring:** latency, token usage, hit rate, citation rate
- ✓ **Evaluation:** answer relevance, faithfulness, citation accuracy
- ✓ **Freshness:** source SLA, last updated, reindex cadence
- ✓ **Safety:** PII redaction, content filters, access control
- ✓ **Scale:** caching, batch ingestion, async reindex, shards
- ✓ **Resilience:** fallbacks, timeouts, partial results

REAL-WORLD EXAMPLES



9 INTERVIEW THINKING

DECISION FLOW



ALWAYS CONSIDER

- Data quality & coverage
- Evaluation plan
- Citations & UX
- Failure modes & fallbacks

10 SENIOR NOTES

- ★ Design for change: schema, sources, and ranking will evolve.
- ★ Invest in evals early: they drive reliable iteration.
- ★ Treat the index like a product: version, test, monitor.
- ★ Optimize end-to-end: retrieval quality > raw LLM power.
- ★ Prefer simple systems you can explain and operate.
- ★ Security & privacy are non-negotiable.



11 MENTAL HOOK

RAG is a **librarian** + a **writer**.



Librarian
finds the right facts (retrieve + rank)



Writer
uses facts to craft a good answer (LLM)



Grounded answer
with citations users can trust.



If you remember one thing

Find → Feed → Generate → Cite → Iterate



Good RAG is about the plumbing, not the model.

RAG Cheatsheet: Build systems that answer with the right facts, at the right time, for the right users.

LLM INFERENCE

1 WHY



Rationale

Turn a trained model into real value.
Fast, reliable, affordable responses
at scale.



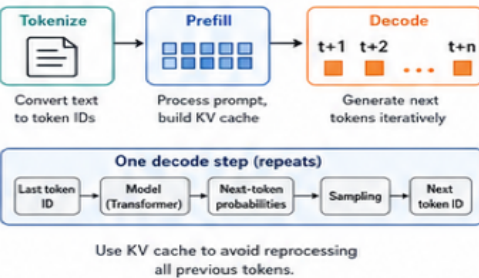
Mental Model

Generate one token at a time.
Reuse past work (KV cache).
Optimize the loop.

Key Goals

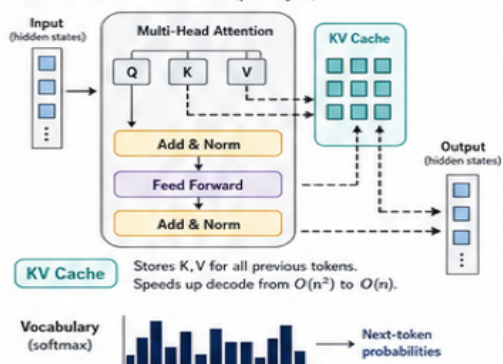
- ✓ Low latency (time to first token, TTFT)
- ✓ High throughput (tokens/sec)
- ✓ High quality (relevance, coherence)
- ✓ Low cost (compute, memory, \$/1K tokens)
- ✓ Reliability & safety

2 CORE IDEA



3 INTERNALS

Transformer Inference (per layer)



4 COMPLEXITY

Compute Complexity (approx)

Phase	What happens	Time Complexity
Prefill (n tokens)	Process full prompt	$O(n^2 \cdot L \cdot d)$
Decode (m tokens)	Generate m tokens	$O(m \cdot n \cdot L \cdot d)$

n = prompt length, m = new tokens, L = layers, d = hidden size

Memory (approx)

Model Weights	$O(\# \text{params} \cdot \text{bytes/param})$
KV Cache	$O(n \cdot L \cdot d \cdot \text{bytes_per_kv})$
Activations (temp)	$O(\text{batch} \cdot L \cdot d)$

Cost Model

Cost $\approx$ (prompt_tokens + generated_tokens) $\times$ (\$ per 1K tokens)



5 TRADE-OFFS

Dimension	Optimize For	Trade-Off
Latency (TTFT)	Smaller model, quantization, prefill parallelism	Lower quality, less context, less accuracy
Throughput	Batching, continuous batching	Higher latency per request, more variance
Quality	Larger model, higher precision, longer context	Higher cost, more memory, slower
Memory	Quantization, KV cache mgmt, shorter context	Quality loss, context loss, more evictions
Cost	Smaller model, quantization, better batching	Potentially lower quality or higher latency

Choose the right point on the Pareto frontier for your use case.

6 DISCUSSION

Strengths

- ✓ Flexible, general-purpose generation
- ✓ Few-shot in-context learning
- ✓ Rapidly improving quality

Limitations

- ⚠ Latency can be high
- ⚠ Hallucinations & consistency issues
- ⚠ Context window is finite
- ⚠ Compute & memory intensive

Alternatives / Complements

- RAG (retrieve then generate)
- Fine-tuned smaller models
- Function calling / tools
- Cascaded or hybrid pipelines

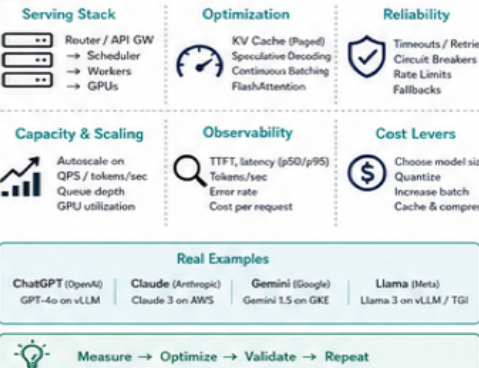
Common Failure Modes

- ⚠ Out-of-memory (context / batch too big)
- ⚠ Slow decode (no KV cache, small batch)
- ⚠ Quality drift (too aggressive quantization)
- ⚠ Unsafe / incorrect outputs

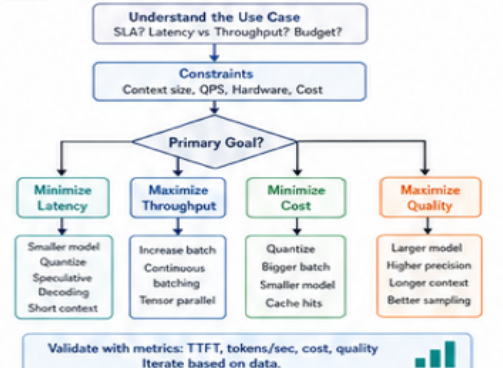
7 RELATED

- Tokenization**: BPE / SentencePiece → token IDs
- Sampling**: Temperature, Top-p, Top-k, Repetition Penalty
- Quantization**: INT8 / FP8 / INT4 / GPTQ / AWQ
- Batching**: Static batching, Continuous Batching
- Parallelism**: Tensor | Pipeline | Data | Expert (MoE)
- Attention Optimizations**: FlashAttention, PagedAttention
- Serving Frameworks**: vLLM, TensorRT-LLM, TGI, SGLang, LiteLLM
- Evaluation**: Perplexity, MMLU, HellaSwag, MT-Bench

8 PRODUCTION



9 INTERVIEW THINKING



10 SENIOR NOTES

- ★ Prefill is quadratic, decode is linear—optimize accordingly.
- ★ KV cache is your friend; manage it (size, eviction, layout).
- ★ Batching is a latency-throughput trade-off.
- ★ Quantization: measure quality (evals) before & after.
- ★ Memory bandwidth > FLOPs for decode; profile!
- ★ Speculative decoding can 2–3x speed up decode.
- ★ Observe tail latency (p95/p99), not just averages.
- ★ Cost is (tokens_in + tokens_out) $\times$ price—track it.
- ★ Safety, privacy, and guardrails are product features.
- ★ Build knobs, not one-size-fits-all systems.

11 MENTAL HOOK



Useful Formulas

Prefill Time $\propto n^2 \cdot L \cdot d$

Decode Time $\propto m \cdot n \cdot L \cdot d$

KV Cache Mem $\propto n \cdot L \cdot d \cdot \text{bytes_per_kv}$

Rule of Thumb

TTFT < 1–2s feels fast

Tokens/sec >> 10 for good throughput

Batch until p95 latency hits your SLA

Remember: Great systems win on the boring loop.

→ **Measure. Optimize. Repeat.**

AI AGENTS

1 WHY

Agents turn reasoning into action.



Solve complex tasks beyond one-shot completion



Use tools & data to interact with the real world



Adapt & persist with memory and feedback



Increase leverage automate multi-step workflows

→ **Rationale:** LLMs know a lot. Agents get things done.

2 CORE IDEA

Mental Model: A goal-driven loop that plans, acts, observes and adapts.



Essence: Plan → Act → Observe → Learn → Repeat until goal is satisfied (or stop safely).

3 INTERNALS

Inside the agent runtime.

1



INSTRUCTIONS

- System prompt, role, rules
- Task spec, success criteria

2



MODEL

- LLM/Reasoner (e.g., GPT-4o)
- Temperature, context window

3



TOOLS

- Functions / APIs / Connectors
- Schemas & examples

4



MEMORY

- Short-term (conversation state)
- Long-term (vector DB / KV)
- Summaries & facts

5



CONTROL LOOP

- Plan → Tool call → Observe
- State update → Next step
- Stop / success / fail

State Contains: messages, tool results, scratchpad, files, counters, permissions, costs

4 COMPLEXITY

What drives cost, latency and reliability.

FACTOR	IMPACT	HOW TO MANAGE
Prompt & Context Size	\$\$\$ Latency ↑	Compress, summarize, retrieve only what's needed
Steps / Iterations	\$\$\$\$ Latency ↑↑	Better planning, early stopping, max steps
Tool Calls	\$\$ Latency ↑	Batching, caching, idempotent design, timeouts
Memory / Retrieval	\$\$ Latency ↑	Index quality, chunking, TTL, freshness
Model Capability	\$\$\$\$ Quality ↑	Right-size model, fallback tiers

Track: Cost per task, p50/p95 latency, success rate, % human interventions.

5 TRADE-OFFS

No free lunch — choose your balance.

MORE OF THIS	IMPROVES	WORSES
Longer Context	Fewer misses, coherence	Cost, latency, distraction
More Tools	Capability, flexibility	Complexity, failure surface
Autonomy	Throughput, hands-off	Risk, cost, debuggability
Strict Guardrails	Safety, predictability	Recall, slower tasks
Stronger Model	Quality, reasoning	Cost, hallucination still possible
More Memory	Personalization, continuity	Staleness, privacy, cost



Dial the mix based on task criticality, cost budget and user trust.

6 DISCUSSION

Alternatives



RAG + Answering

Best for: Q&A, lookup, static knowledge



Workflows / Orchestrators

Best for: Deterministic business processes



Fine-tuned Models

Best for: Narrow domain, stable I/O



Simple LLM (No agent)

Best for: Single-step, low risk tasks

Common Failure Modes



Hallucination

Verify, cite sources, confidence checks



Tool Misuse

Schemas, validations, unit tests



Prompt Injection

Input filtering, sandbox, least-privilege



State Corruption

Checksums, versioned state, retries



Runaway Loops

Step limits, budgets, circuit breakers

What would you pick and why?
Defend with trade-offs + safeguards.

7 RELATED

Patterns & building blocks.



ReAct Interleave reasoning and action.



Plan-and-Execute Separate planner from executor.



Toolformer Model learns when to call tools.



Self-Reflection Critique and improve outputs.



Memory Hierarchy Buffer → Summary → Long-term.



Guardrails Policies, validators, PII filters.



Human-in-the-Loop Approve, edit, or take over.



Combine patterns, don't worship them.
Pick what fits the job.

8 PRODUCTION

What good looks like in prod.



Permissions

Least-privilege API keys, scoped actions, RBAC



Retries & Timeouts

Exponential backoff, idempotency keys, jitter



Human Approval

High-risk actions gated, review UI, audit log



Observability

Traces, logs, metrics, tool logs, token usage



Evaluation

Offline evals, golden sets, online A/B, win rate



Failure Containment

Rate limits, budgets, circuit breakers, kill switch



Design for failure first.
Safety, visibility, and control.

9 INTERVIEW THINKING

A decision flow.



Always ask:

- What can go wrong?
- How will we measure success?
- What's the rollback plan?

10 SENIOR NOTES

Senior engineer / tech lead judgment.



- Start simple; ship learning, then iterate.
- Define success metrics early.
- Own the prompt + tool contract like code.
- Prefer predictability over cleverness.
- Design for operability and handoffs.
- Cost is a feature—budget and monitor.
- Document assumptions & failure modes.
- Security & privacy are non-negotiable.

Good answers show:

- ★ Systems thinking (end-to-end)
- ★ Risk awareness & mitigation
- ★ Trade-off articulation
- ★ Bias for iteration with measurement

11 MENTAL HOOK

One line to remember.



An agent is a **REASONING ENGINE** with **HANDS** and **MEMORY**



GOAL
What we want

REASON
Think & plan

ACT
Use tools

OBSERVE
Get feedback

LEARN
Update state

REPEAT
Until done

Keep it safe. Make it useful. Make it measurable.

BUILD WITH JUDGMENT. </>

SHIP WITH CONFIDENCE. 🚀

LEARN AND ITERATE. 🔄

AI GUARDRAILS

1 WHY

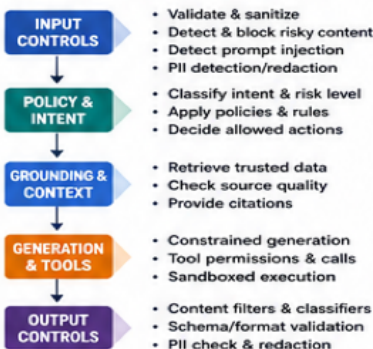
RISK IS INEVITABLE. GUARDRAILS MAKE AI USEFUL, SAFE AND TRUSTED.

- Protect users & data**
Prevent harm, leakage and misuse
- Meet policy & compliance**
PII, policy, legal, industry requirements
- Improve reliability**
Reduce errors, hallucinations and off-policy behavior
- Enable safe autonomy**
Use tools and data sources with controlled permissions
- Build trust & auditability**
Observe, explain and continuously improve

Goal: Maximize helpfulness while minimizing risk.

2 CORE IDEA

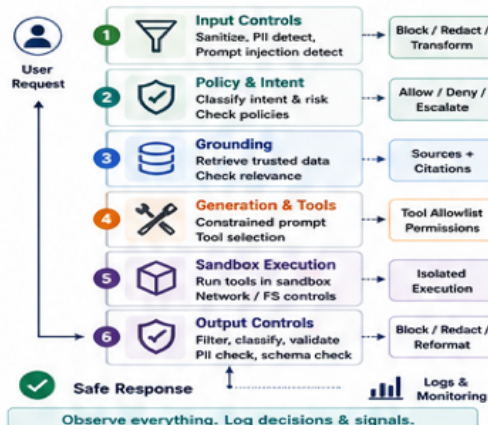
LAYERED CONTROLS ACROSS THE LIFECYCLE.



Defense in depth: layers assume failure of others.

3 INTERNALS

TYPICAL GUARDRAILED REQUEST FLOW



4 COMPLEXITY

DRIVERS: LAYERS, MODELS, INTEGRATIONS, SCALE



Components

L layers, M models,
T tools, R rules



Time Complexity
(per request)

$O(L \cdot (C_m + C_r + C_t))$
 C_m : model cost
 C_r : retrieval cost
 C_t : tool cost



Space Complexity

$O(\text{Index} + \text{Logs} + \text{Cache} + \text{Models})$



Cost per 1K requests (est.)

$C_{\text{total}} = \sum \text{layer costs} + \text{tokens} + \text{infra}$

REFERENCE COST TIERS
(ROUGH RANGE)

Tier	Cost / 1K req (USD)
Light	\$0.20 – \$1
Standard	\$1 – \$5
Heavy	\$5 – \$20+

EXAMPLE (STANDARD)

- 2 LLM calls
- 1 retrieval
- 2 classifiers
- 1 tool (sandboxed)

$\approx \$1 - \$5 / 1K \text{ req}$

Measure each layer. Optimize the expensive ones.

5 TRADE-OFFS

Dimension	More Safety	More Freedom
Latency	Slower (more checks, retrieval, models)	Faster
Cost	Higher	Lower
Accuracy (Helpfulness)	May be lower (precision > recall)	May be higher (recall > precision)
Coverage	Broader risks covered	Narrower
User Experience	More friction (blocks, asks)	Smoother
Explainability	Better (more signals & logs)	Less
Autonomy	More constrained (actions limited)	More autonomous (actions allowed)

Tune to your risk tolerance and user expectations.

6 DISCUSSION

- Key Trade-Offs**
 - Safety vs. helpfulness
 - Coverage vs. latency/cost
 - Automation vs. control
- Alternatives**
 - Human-in-the-loop
 - Retrieval-only (no gen)
 - Fine-tuned / smaller models
- Failure Modes**
 - Prompt injection / jailbreaks
 - Policy bypass via tools
 - PII leakage / re-identification
 - Hallucinations / wrong sources
 - Sandbox escape
 - Over-blocking (false positives)

Design for known risks. Monitor for unknown ones.

7 RELATED

- RAG (Retrieval Augmented Generation)**
Ground with trusted, cited data
- Content Moderation**
Toxic, harmful, illegal content detection
- PII Management**
Detect, redact, minimize, retain safely
- Prompt Injection Defense**
Detect, neutralize, isolate instructions
- Access Control (ABAC/RBAC)**
Who can use what, and how
- Tool & Function Calling**
Capabilities, parameters, rate limits
- Sandboxing & Isolation**
Process, network, file system isolation
- Observability & Audit**
Logs, traces, metrics, evaluations

8 PRODUCTION

REAL-WORLD EXAMPLES

E-COMMERCE ASSISTANT Guardrails: PII redaction, policy checks, RAG product data, tool allowlist (order lookup), output schema validation.	Result: ↓ CSAT issues ↓ PII leaks ↑ Trust
FINANCIAL COPILOT Guardrails: Intent classification, policy engine, citations from approved docs, sandboxed calculations.	Result: ↓ Hallucinations ↓ Policy violations ↑ Accuracy
HEALTHCARE TRIAGE Guardrails: PHI redaction, medical knowledge retrieval, strict refusal policies, human escalation.	Result: ↓ Risk exposure ↑ Safety ↑ Adoption

Start small. Add layers as risks and scale grow.

9 INTERVIEW THINKING

DECISION FLOW



Think like a product + risk + systems owner.

10 SENIOR NOTES

- Start with Risk Taxonomy
Business, safety, legal, privacy, security.
- Quantify & Prioritize
Impact × Likelihood → Focus efforts.
- Layer, Don't Rely
No single control is sufficient.
- Fail Closed on High Risk
Prefer safe denial + human review.
- Observability First
You can't improve what you don't measure.
- Continuously Validate
Offline evals + online monitoring + red-teaming.

Senior engineers reduce risk while enabling velocity.

11 MENTAL HOOK

THE CASTLE & MOAT



Assume breach. Build layers. Log everything. Improve always.

QUICK CHECKLIST

- Input validation & prompt injection detection
- PII detection, minimization, redaction
- Policy & intent classification
- Grounding with trusted, cited sources
- Tool allowlist, permissions, rate limits
- Sandbox isolation for tool execution
- Output filtering, schema validation
- Monitoring, alerting, audits, evals
- Incident response & escalation path

PROMPT ENGINEERING

1 WHY

Good prompts → reliable, useful, safe outputs.
Your leverage is in the prompt.

- Align model to your goal
- Reduce errors, hallucinations, and unsafe outputs
- Increase consistency, controllability, and speed
- Lower cost through less iteration and clearer outputs

Rationale: LLMs are general. Prompts provide intent, context, constraints, and structure.

4 COMPLEXITY

- Drivers:**
- Context length (tokens)
 - Reasoning steps / depth
 - Output length & structure
 - Tools / API calls
 - Examples & few-shot count



Rule of Thumb (per 1K tokens)

Input	Output	Latency Impact
Small (< 1K)	Short (< 200)	Low
Medium (1K-4K)	Medium (200-1K)	Moderate
Large (4K-16K)	Long (> 1K)	High
Very Large (16K+)	Very Long (> 2K)	Very High

Cost = (Input Tokens × \$/1K) + (Output Tokens × \$/1K) + Tool Calls × Cost

7 RELATED

- Chain-of-Thought (CoT)**
Ask the model to reason step-by-step.
- Tree of Thoughts (ToT)**
Explore multiple paths, pick the best.
- RAG (Retrieval)**
Retrieve relevant docs, then answer.
- Function Calling / Tools**
Let the model call tools/APIs.
- Guardrails & Policies**
Enforce safety and compliance.
- Eval Frameworks**
Measure quality, regressions, and cost.

These are patterns. Prompts orchestrate them.

10 SENIOR NOTES

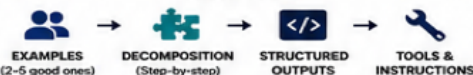
- Think in systems, not prompts. Prompts + data + tools + evals.
- Optimize end-to-end (quality, cost, latency, safety).
- Design for change: model swaps, scale, new data.
- Own the risk: privacy, bias, compliance.
- Build feedback loops from users and evals.
- Document decisions and assumptions.

Senior engineers create durable, observable, and safe outcomes.

2 CORE IDEA

- TASK**
Clear objective. Use strong verbs.
e.g., Summarize the customer feedback.
- CONTEXT**
Give background, relevant data.
e.g., Product is a billing platform for SaaS companies.
- CONSTRAINTS**
Rules, scope, style, limits, assumptions.
e.g., Only use provided data. Be concise. Max 120 words.
- OUTPUT**
Format, length, tone, audience.
e.g., Bullet points in Markdown. For execs.

BOOSTERS



Golden rule: Be explicit so the model has less to guess.

5 TRADE-OFFS

Choice	↑ More of This	↓ Less of This
Specificity	Accuracy, relevance, consistency	Flexibility, creativity
Examples	Better pattern following	Higher token cost
Decomposition	Reliability on complex tasks	More latency, tokens
Structured Outputs	Parseability, automation	Less natural language
Grounding (Data)	Factuality, trust	Context length, staleness
Guardrails	Safety, robustness	May block edge cases

Balance based on goal, risk, cost, and user experience.

8 PRODUCTION

- Version prompts**
Track changes, roll back.
- Templates & Variables**
Keep prompts DRY and reusable.
- Observability**
Log prompts, outputs, latency, cost.
- Evaluation**
Golden set + automated metrics.
- Safety & Injection Defense**
Input filters, delimiters, allowlists.
- Fallbacks**
Retry, simpler prompt, human-in-loop.
- Cost Controls**
Token budgets, caching, batching.
- Continuous Improvement**
Analyze, iterate, A/B test.

Ship → Measure → Learn → Improve

11 MENTAL HOOK



GUIDE THE THINKING
SHAPE THE OUTPUT

3 INTERNALS

Mental model: The model predicts the next token.

Typical internal workflow:

- 1 READ & PARSE**
Understand instruction, context, and constraints.
- 2 PLAN**
Form an approach, recall knowledge, choose tools.
- 3 EXECUTE**
Generate tokens (draft). Use tools if instructed.
- 4 REFLECT & REFINE**
Check constraints, self-correct, finalize output.

Your job: Shape the inputs to guide each step.

6 DISCUSSION

Alternatives:

- Fine-tuning / RAG / Tool-first architectures
- Programmatic post-processing
- Smaller models / Specialized models

Failure Modes:

- Ambiguous intent → Off-target answers
- Missing constraints → Verbose or unsafe output
- Outdated knowledge → Stale or wrong facts
- Prompt injection → Policy bypass / jailbreak
- Over-complex prompt → Confusion, drift, errors

Mitigations:

Clarify, constrain, ground, structure, iterate, evaluate.

9 INTERVIEW THINKING



BE EXPLICIT. GUIDE THE MODEL. MEASURE EVERYTHING. ITERATE RELENTLESSLY.

MODEL EVALUATION

1 WHY



Rationale: Ensure models are useful, safe, reliable, and cost-effective before and after launch.



Protect users & brand

Safety, fairness, privacy, compliance



Drive quality & trust

Better decisions, higher adoption



Manage cost & latency

Deliver value within constraints



Prevent regressions

Catch issues early, ship with confidence



Measure what matters. Improve what you measure. Ship what you trust.

2 CORE IDEA

Mental model:

Measure predicted behavior against ground truth (or goals) with the right mix of automation, humans, and statistics.

OFFLINE (Before launch)	ONLINE (After launch)
Fixed datasets	Live traffic
Cheap & fast	Real user outcomes
Reproducible	A/B & canary
Detect regressions	Detect drift
Gate releases	Guide improvements

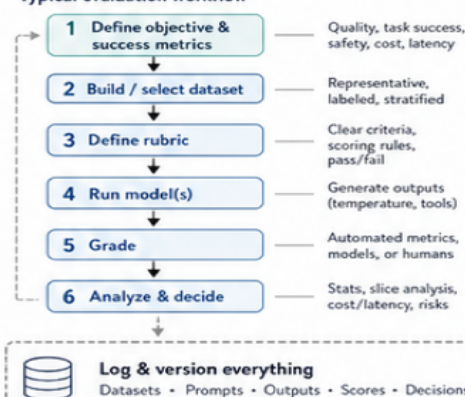
What we evaluate

Quality	Task Success	Safety	Efficiency (Latency/Cost)
---------	--------------	--------	---------------------------

Right data → Right rubric → Right grader → Right stats → Trustworthy decisions

3 INTERNALS

Typical evaluation workflow



4 COMPLEXITY

Evaluation cost drivers

Dataset size (N items)	$O(N)$	•••
Output length (tokens)	$O(N \times L)$	•••
Number of models (M)	$O(M \times N \times L)$	•••
Grader type		
• Rule/Heuristic	$O(N)$	•••
• LLM-as-judge	$O(N \times L_g)$	•••
• Human review	$O(N \times H)$	•••
Human time (H)	$O(N \times H)$	•••
Statistical rigor	$O(N \log N)$	•••
Online traffic	$O(QPS \times T)$	•••

Cost: • Low •• Medium ••• High

Total Eval Cost = $(M \times N \times L) \times C_{\text{model}} + N \times C_{\text{grader}} + N \times H \times C_{\text{hr}}$
(Model inference + Grading + Human review)

5 TRADE-OFFS

Optimize for...	Pros	Cons
Automation (LLM/Rubric)	Scale, speed, consistent	May miss nuance, grade drift
Human Review	Nuance, catch edge cases	Expensive, slow, less scalable
Offline Rigor	Cheap, reproducible, prevents regressions	May not reflect real world
Online Metrics	Real outcomes, detect drift	Noisier, slower feedback
Broad Coverage (large dataset)	Detects more failure modes	High cost & longer runs
Deep Evaluation (per item depth)	High signal, better diagnosis	Fewer items, less coverage
Lower Latency	Better UX, higher conversion	May reduce quality
Lower Cost	Higher margins, scale sustainably	May limit model choice/quality

Balance for the phase: pre-launch quality vs. post-launch learning.

6 DISCUSSION

Key topics to cover

- Offline vs. online evaluation and when to use each
- Dataset quality: representativeness, freshness, bias
- Rubric design: dimensions, weights, pass/fail rules
- Graders: heuristic metrics, model graders, humans
- Metrics: precision/recall, task success, safety, latency, cost
- Statistical rigor: significance, CIs, power, multiple tests
- Regression testing & thresholds for release
- Slicing: by cohort, prompt type, locale, length, tools
- Failure analysis → fixes → re-evaluate (iterate)
- Guardrails: red teaming, jailbreaks, PII, toxicity
- Reporting & dashboards for decisions

Common failure modes



7 RELATED

Alternatives & complements

- A/B Testing (online)**
Best for user outcomes and product metrics
- Canary / Shadow Deployments**
Gradual rollout, compare to control
- Red Teaming**
Finds risks adversaries might exploit
- Synthetic Data / Generation**
Scale coverage, but validate against real data
- Observability & Monitoring**
Drift, errors, latency, cost, guardrail hits
- Human-in-the-loop Systems**
Use feedback to improve models & graders

Evaluation is a system: data → metrics → decisions → learning

8 PRODUCTION

Real-world evaluation in practice

Shopify	• Offline eval suite for quality, safety, policy • Golden sets + regression gates before release • Human review for edge cases
Anthropic (Claude)	• Constitutional AI evaluations & red teaming • Safety evals across harm categories • Ongoing model + grader improvements
Google Search	• Offline: NDCG, relevance, freshness • Online: interleaving, A/B for user satisfaction • Continuous eval & quality dashboards
OpenAI (ChatGPT)	• Pre-launch offline evals on quality & safety • Post-launch monitoring, user reports, mitigation • Regression testing each model iteration

Pattern: Offline gates → Controlled rollout → Online learning loop

9 INTERVIEW THINKING

How to approach any eval question



10 SENIOR NOTES

Senior judgment

- ✓ Define success in business terms, then map to metrics
- ✓ Invest in high-quality datasets & rubrics early
- ✓ Prefer simple, stable metrics over fragile proxies
- ✓ Use multiple metrics—no single number tells the story
- ✓ Slice everything: cohorts, tasks, locales, lengths, tools
- ✓ Quantify uncertainty; don't over-claim results
- ✓ Set clear thresholds & escalation paths
- ✓ Build evals as product: versioned, tested, observable
- ✓ Create feedback loops from production to training/evals
- ✓ Know when to stop evaluating and ship

Statistical rigor checklist

- Test type**
Paired, stratified where possible
- Significance level**
 $\alpha = 0.05$ (or business-justified)
- Effect size**
Report absolute & relative lift
- Confidence intervals**
95% CI for key metrics
- Power / Sample size**
Compute before running tests
- Multiple comparisons**
Control FDR or use correction
- Practical significance**
Is lift meaningful to users?
- Report**
Metric, lift, CI, p-value, N, slices

Release gate example

Quality (LLM-as-judge)	≥ 0.85
Task Success (exact/functional)	$\geq 90\%$
Precision / Recall	$P \geq 0.85 \quad R \geq 0.80$
Safety (harm rate)	$\leq 0.5\%$
PII Leakage	0 tolerance
Latency p95	$\leq 2.5 \text{ s}$
Cost / 1k tokens	$\leq \$0.02$
Regression vs. previous	Within -2%

All gates pass? → Ship ✓

11 MENTAL HOOK

Evaluate like a scientist, ship like a product manager.



What gets measured gets improved.
What gets improved ships value.
Measure → Learn → Ship → Impact

GREAT DATA • CLEAR RUBRICS • RELIABLE GRADERS • STATISTICAL RIGOR • BETTER DECISIONS • USER VALUE

AI ARCHITECTURE

1 WHY



RATIONALE

Deliver reliable, useful, and safe AI experiences at scale—balancing quality, latency, cost, and risk.



MENTAL MODEL

Requests flow through a control plane that routes, augments, and governs model calls using data, tools, and policies.

KEY OUTCOMES

- ✓ Right model, right context, right time
- ✓ Safe, compliant, and explainable
- ✓ Observable and cost-efficient
- ✓ Continuously improving

2 CORE IDEA

THE PATTERN

Compose capabilities—models, data, tools, and policies—via modular services. Orchestrate the workflow; retrieve and reason over data; enforce guardrails; evaluate and learn.



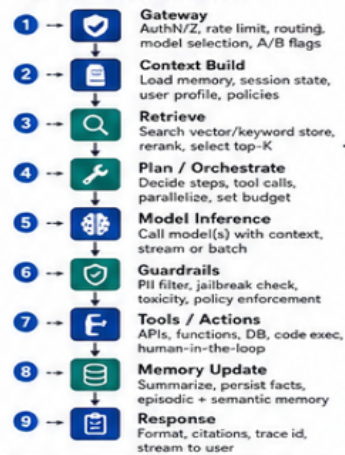
BUILDING BLOCKS



DATA GOVERNANCE across all layers
Privacy • Security • Lineage • Access • Retention

3 INTERNALS

TYPICAL WORKFLOW



4 COMPLEXITY

TIME & SPACE (Big-O) — per request

Component	Time Complexity	Space
Gateway & Routing	$O(1)$	$O(1)$
Retrieval (Vector Search)	$O(\log N) \sim O(N^{\alpha})$	$O(N \cdot d)$
Rerank (top-K)	$O(N \log K)$	$O(K \cdot d)$
Orchestration (Steps)	$O(S)$	$O(S)$
Tool Calls	$O(T)$	$O(T)$
Model Inference	$O(\text{Tok}_{in} + \text{Tok}_{out})$	$O(\text{Tok}_{ctx})$
Guardrails	$O(\text{Tok})$	$O(1)$
Memory Update	$O(M)$	$O(M)$

COST MODEL (per request)

- Total Cost** $\approx C_{model} + C_{retrieval} + C_{tools} + C_{overhead}$
- $C_{model} = f(\text{tokens}_{in}, \text{tokens}_{out}, \text{model_price})$
 - $C_{retrieval} = f(\text{index_size}, \text{topK}, \text{provider_price})$
 - $C_{tools} = f(\# \text{calls}, \text{external API pricing})$
 - $C_{overhead} = \text{infra, observability, storage, egress}$

LATENCY BUDGET (p95 target)

Network	Retrieval	Model	Other
10~20%	20~30%	50~70%	10~20%

Goal: keep p95 latency within SLO while minimizing cost.

5 TRADE-OFFS

Dimension	Option A	vs	Option B	Trade-Off
Quality	Larger model	vs	Smaller model	Quality ↑ vs Cost/Latency ↑
Latency	Sync (blocking)	vs	Async/Stream	Simplicity vs Perceived speed
Cost	Higher context (Top-K ↑)	vs	Lower context (Top-K ↓)	Recall vs Cost ↓
Freshness	Real-time data	vs	Cached/Batch	Freshness vs Cost/Latency ↓
Safety	Strict guardrails	vs	Flexible guardrails	Safety ↑ vs False positives ↑
Reliability	Single provider	vs	Multi-provider (failover)	Cost/simple vs Resilience ↑
Complexity	Orchestration	vs	Direct prompt	Flexibility vs Build complexity
Memory	Long-term + short-term	vs	Short-term only	Personalization vs Privacy/Cost ↑

COMMON TRADE-OFF HEURISTIC

- Start simple → Measure → Optimize the bottleneck, not everything.

6 DISCUSSION

ALTERNATIVES

- Prompt-only (no retrieval): simple, low cost, low factuality
- Fine-tuned model: high quality, high cost, slow to adapt
- Agents vs Workflows: flexible vs predictable
- Vector DB vs SQL/Graph: semantic vs structured precision
- Hybrid Search (BM25 + Vector): recall + precision
- Human-in-the-loop: quality & safety for high-risk tasks

FAILURE MODES

- ✗ Hallucinations / outdated info
- ✗ Prompt injection / jailbreak
- ✗ PII leakage / policy violations
- ✗ Tool misuse / unsafe actions
- ✗ Context overflow / lost in the middle
- ✗ Latency spikes / rate limits
- ✗ Cost overruns / runaway loops
- ✗ Data drift / retrieval misses

MITIGATIONS

- ✓ RAG + citations
- ✓ Guardrails + red teaming
- ✓ PII detection + masking
- ✓ Tool allowlist + approvals
- ✓ Summarization + chunking
- ✓ Caching + timeouts + bulkheads
- ✓ Budgets + circuit breakers
- ✓ Monitoring + evals + alerts

7 RELATED

PATTERNS & ARCHITECTURES

- RAG (Retrieval Augmented Generation)
- Tool-Using Agent / Function Calling
- Plan-and-Execute / ReAct
- Router / Multi-Model Routing
- Event-Driven Orchestration
- CQRS for Read (RAG) vs Write (Tools)
- Human-in-the-Loop Pipeline

KEY CONCEPTS

- Tokens • Embeddings • Vector DB • Chunking
- Semantic Search • Re-ranking • Memory
- Guardrails • Evaluations • Tracing
- Caching • Rate Limiting • Circuit Breaker

10 SENIOR NOTES

- Optimize the bottleneck, not the average. Measure p50/p95, cost per request, and quality.
- Design for change and failure. Providers, models, and prices will change.
- Guardrails are product features. Make safety observable and configurable.
- Data is your moat. Invest in curation, freshness, and lineage.
- Prefer explicit over implicit. Version prompts, policies, and data.
- Keep the user in the loop. Escalate high-risk, unclear, or high-impact actions.

SENIOR JUDGMENT

- ✓ Where is the real risk?
- ✓ What is "good enough"?
- ✓ What can we simplify?
- ✓ What will not scale?
- ✓ What's our escape hatch?

8 PRODUCTION

REAL-WORLD EXAMPLES

- Customer Support Copilot**
RAG over docs + tools for order status, refunds
Impact: ~35% AHT
Cost: \$0.004 / turn
- Developer Assistant**
Code + docs retrieval, tool execution
Impact: +20~30% dev velocity
- Enterprise Search**
Hybrid search + rerank + citations
Impact: +2~3x employee productivity
- Compliance Q&A**
Policy retrieval + strict guardrails
Impact: 0 policy violations

PRODUCTION CHECKLIST

- ✓ SLOs defined (latency, quality, cost)
- ✓ Cost budgets & alerts
- ✓ Caching strategy in place
- ✓ Failbacks + multi-provider
- ✓ Guardrails & PII handling
- ✓ Evaluation suite (offline + online)
- ✓ Observability & tracing
- ✓ Canary + rollback
- ✓ Data governance enforced
- ✓ Runbooks & dashboards

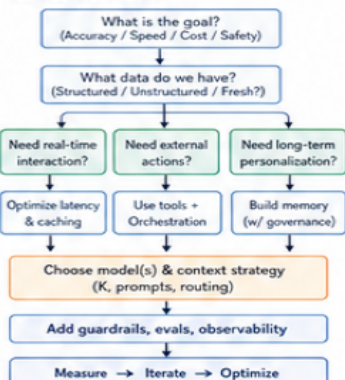
11 MENTAL HOOK

"START SIMPLE. ADD INTELLIGENCE. ADD SAFETY. MEASURE EVERYTHING."



9 INTERVIEW THINKING

DECISION FLOW



Always check:

- ✓ Latency (p95)
- ✓ Cost per request
- ✓ Quality (Evals)
- ✓ Safety (Policies)
- ✓ Reliability (Errors)

PROMPT TEMPLATE

1 WHY



Create a reusable prompt contract that is clear, safe, predictable, and easy to evolve.

RATIONALE

- ✓ Consistency across use cases
- ✓ Higher quality outputs
- ✓ Easier debugging & iteration
- ✓ Safer, more reliable behavior
- ✓ Scalable across teams

SUCCESS METRICS

- ✓ Accuracy / Relevance ↑
- ✓ Hallucinations / Errors ↓
- ✓ Reusability / Portability ↑
- ✓ Latency & Cost ↓

4 COMPLEXITY

MEASURABLE COMPLEXITY & COST

COMPLEXITY DRIVERS

- Input size (tokens)
- Context breadth
- Reasoning depth
- Tool calls
- Branching / Iteration
- Output size

ESTIMATE FRAMEWORK

$$\text{Cost} \approx (I + C + R + T + O) \times P \times U$$

I = Input tokens
C = Context breadth factor
R = Reasoning depth factor
T = Tool call count (est.)
O = Output tokens (est.)
P = Model price / 1K tokens
U = Uncertainty / retry factor

ORDER OF GROWTH (LOW → HIGH)



Measure, don't guess. Track actuals vs estimates.

7 RELATED

RELATED PATTERNS & CONCEPTS

- System Prompting**
Set behavior & boundaries
- BAG + Grounding**
Add facts, reduce hallucinations
- Function Calling / Tools**
Extend capabilities safely
- Self-Consistency**
Try multiple paths, vote
- Structured Outputs**
Schemas, JSON, validation
- Guardrails & Policies**
Safety, compliance, limits

Know the toolbox. Combine wisely.

10 SENIOR NOTES

SENIOR ENGINEER / TECH LEAD JUDGMENT

- Clarity over cleverness**
Prefer explicit, boring, predictable.
- Design for change**
Version, modularize, halloween.
- Obscurity by default**
Log prompts, inputs, outputs, metrics.
- Safety is non-negotiable**
Least privilege, guardrails, evals.
- Cost discipline**
Measure, set budgets, fail safe.
- Empower the team**
Reusable templates, docs, examples.

SENIOR CHECKLIST

- ✓ Is it correct?
- ✓ Is it robust?
- ✓ Is it safe?
- ✓ Is it observable?
- ✓ Is it cost-effective?
- ✓ Is it maintainable?
- ✓ Will it scale?

Ship reliable systems. People trust consistency.

2 CORE IDEA

MENTAL MODEL

A prompt is a contract.



THE CONTRACT COVERS

- Role
- Objective
- Context
- Inputs
- Constraints
- Procedure
- Tools
- Examples
- Output Schema
- Uncertainty
- Safety
- Versioning

Be explicit. Remove ambiguity. Verify outputs.

5 TRADE-OFFS

COMPARE APPROACHES

Approach	Quality	Cost	Latency	Control	Robustness
Single Prompt (One-shot)	★★	★★★	★★★	★	★
Few-shot Examples	★★★	★★	★★	★★	★★
Chain-of-Thought (Structured)	★★★★	★	★	★★★	★★★
ReAct / Tool-use (Iterative)	★★★★	★	★	★★★★	★★★★
Multi-Agent (Orchestrated)	★★★★	★	★	★★★★	★★★★

KEY TRADE-OFFS

- Quality → Cost
- Latency → Depth
- Control → Flexibility
- Simplicity → Reliability

WHEN TO CHOOSE

- Need speed & low cost → Simple
- Need accuracy & reasoning → CoT
- Need real-world actions → Tools
- Need scale & specialization → Agents

Optimize for the goal, not the trend.

8 PRODUCTION

REAL PRODUCTION EXAMPLES

	Primary Goal	Key Tools
Customer Support Copilot	Accurate answers fast	RAG, Tools (lookup, CRM)
Code Assistant	Generate & review high-quality code	Repo search, Linters, Tests
Data Analyst	Analyze data & explain insights	SQL, Python, Charts
Content Writer	Draft, edit & refine content	RAG, Templates, Grammar tools
Research Assistant	Summarize & synthesize research	Web search, RAG, Notes
Ops / Incident Triage	Triage & recommend actions	Logs, Metrics, Runbooks

Real value = reliable outcomes at scale.

11 MENTAL HOOK

ONE MEMORABLE HOOK



A GREAT PROMPT IS A CRISP CONTRACT BETWEEN HUMANS AND MODELS.



DEFINE
Clearly



GUIDE
Step-by-step



CONSTRAIN
Boundaries



DELIVER
Value

Clarity + Constraints = Consistent Outcomes

3 INTERNALS

INTERNAL WORKFLOW



6 DISCUSSION

ALTERNATIVES

- Retrieval Augmented Generation (RAG)
- Fine-tuned / Domain-specific model
- Rules / Deterministic system
- Human-in-the-loop
- Hybrid approaches

POTENTIAL FAILURE MODES

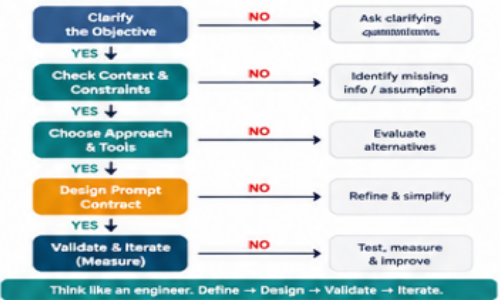
- ⚠ Ambiguous instructions → off-target
- ⚠ Insufficient context → incomplete / wrong
- ⚠ Overconfident answers → hallucinations
- ⚠ Tool misuse / errors → bad actions
- ⚠ Schema drift → parsing failures
- ⚠ Security / policy violations → unsafe output
- ⚠ Cost blowout → runaway loops

DISCUSSION CHECKLIST

- ✓ Is the objective clear and measurable?
- ✓ Is the context sufficient and current?
- ✓ Are constraints realistic and enforced?
- ✓ Are tools necessary and well-defined?
- ✓ Is the output schema precise and testable?
- ✓ How will we handle uncertainty & errors?
- ✓ What are the safety guardrails?

9 INTERVIEW THINKING

INTERVIEW DECISION FLOW



VERSIONING & CHANGE MANAGEMENT

- Semantic Versioning:** MAJOR.MINOR.PATCH
Example: 1.2.0
- Change Log:** Document what changed & why
- Backward Compatibility:** Avoid breaking changes
- Deprecation Policy:** Announce & sunset cleanly
- Testing:** Regression tests for prompts & outputs
- Rollout:** Canary → Monitor → Full release

Version it. Test it. Trust it.

CONTRACT IT. → DESIGN IT. → MEASURE IT. → IMPROVE IT. → SCALE IT.